

Javaプログラミング入門

Introduction to Java Programming

第11回 例外処理・ファイル操作

Class 11. Exception Handling, File I/O

早稲田大学 基幹理工学部 情報通信学科 講師

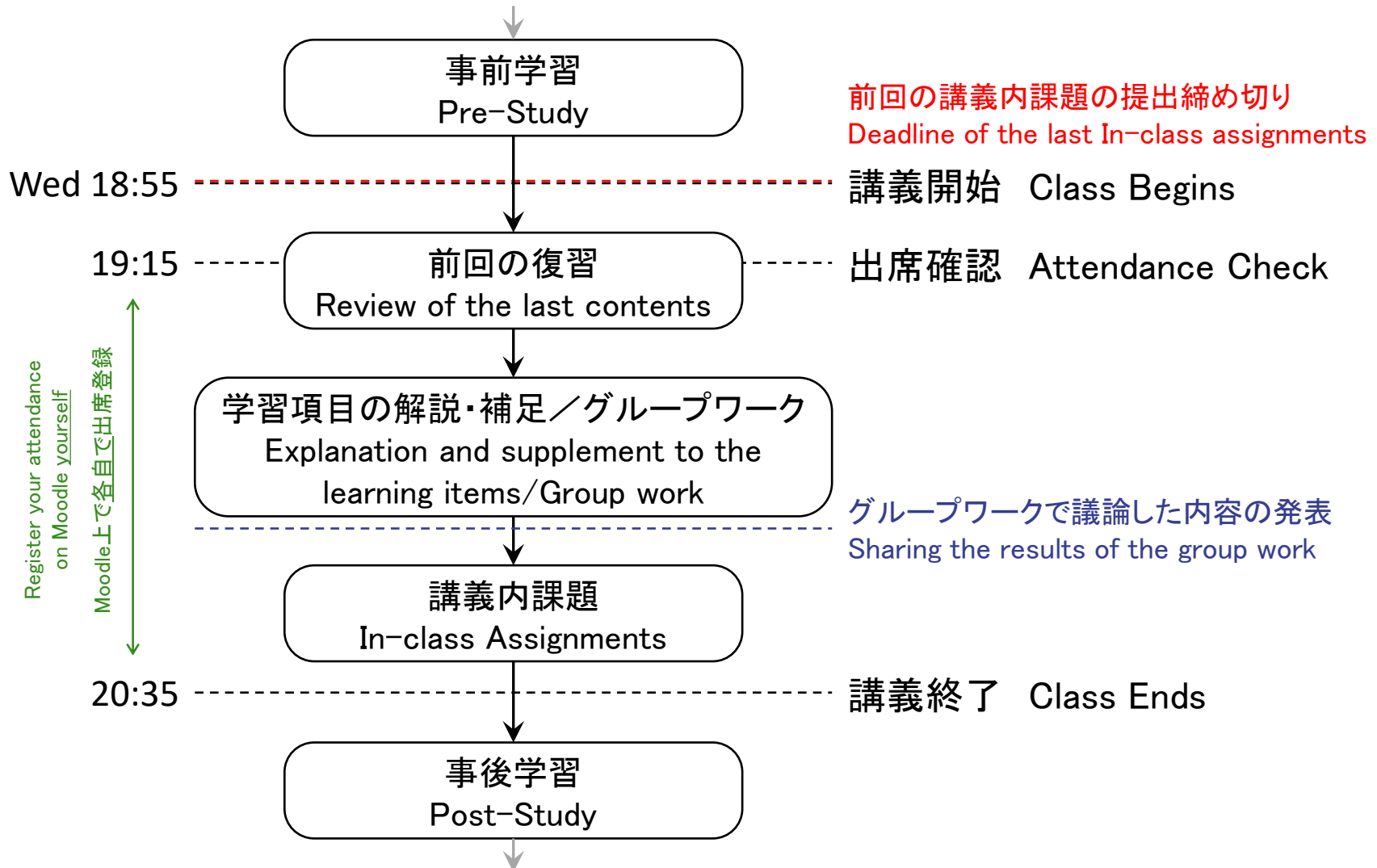
吉井 一駿

Kazutoshi YOSHII

講義の流れ Flow of the Class

本講義は反転講義です。Moodle上にある予習ビデオを視聴の上講義に臨むこと。

This course is a flipped classroom. Watch the video before each class.



第11回講義 Class 11

例外処理・ファイル操作 Exception Handling, File I/O



第10回 例外処理・ファイル操作 00:06:32



Part10 Exception handling and File I/O 00:05:30

到達目標：下記キーワードの理解

Goals for the Next Class: Understanding the Following Topics

例外処理 ExceptionHandling ファイル入出力 File
Input/Output コマンドライン引数 CommandLineArguments

期末レポートについて About Final Report

シラバスの授業計画では、

第13回 グラフィックスの描画:ラベル・テキスト

第14回 プログラミング演習(第13回までの内容) を予定しています。

もし皆さんからの不服が無ければ、

第13回の講義の終わりに最終レポートの課題を提示して、

第14回の講義は課題の解説に充てようと思いますがいかがでしょうか。

問題があれば、7月3日までにメールで連絡を下さい。

We have planned

Class 13: Graphics: Labels, Text

Class 14: Programming Exercise (Review contents upto Class 13)

in the Course Schedule on Syllabus.

If there are no objections, I am considering announcing the final report assignment at the end of Class-13 and using Class-14 for an explanation of the assignment.

If you have any concerns, please email me by July 3.

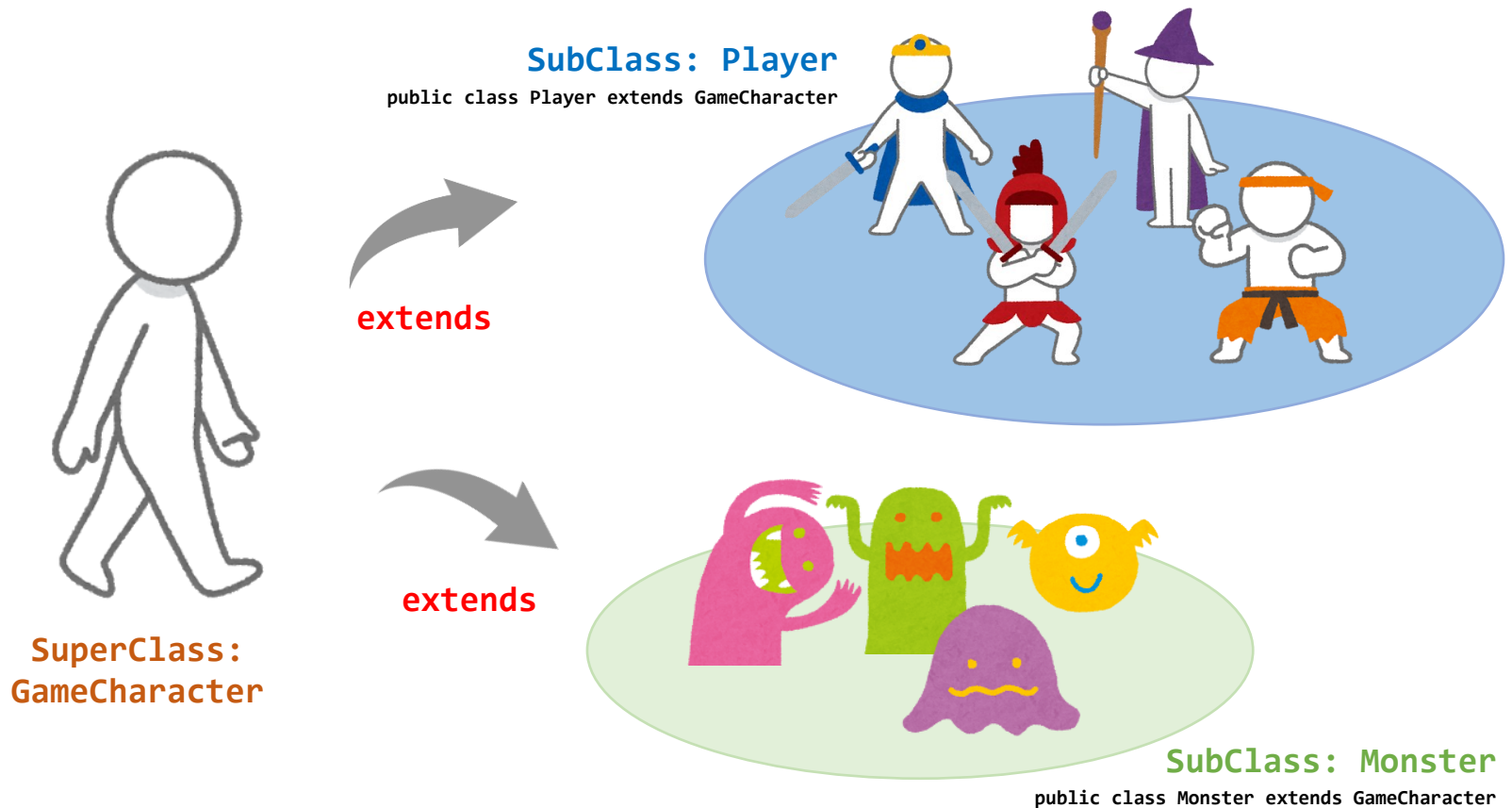


前回の復習

Review of the previous class

継承 Inheritance

- サブクラスが、スーパークラスのフィールド・メソッドを引き継ぐ
Subclass inherits the fields and methods from a superclass.
- コンストラクタはサブクラスに継承されない
Constructors are not inherited to subclass.



スーパークラス Superclass

- スーパークラスはクラスがextendされたもので、単なるクラスである。
Superclass is an ordinary class.
When class is extended, the class call as Superclass.
- したがって、特別に宣言をする必要はない。
Therefore, no special declaration is required in the superclass.

スーパークラス Superclass



フィールド Field

名前：早稲田太郎 Name: Taro Waseda

誕生年：1999 BirthYear: 1999

メソッド Method

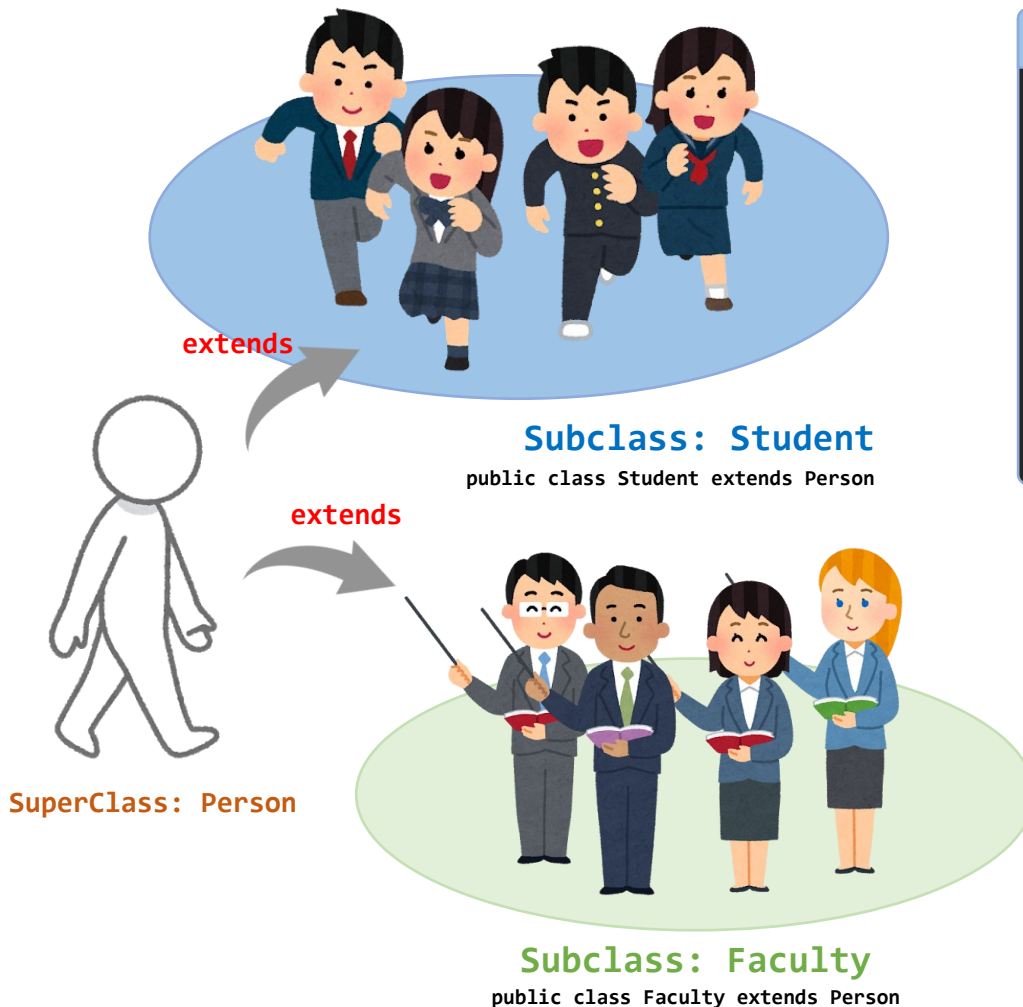
Talk(), Walk(), ...

Person.java (Superclass, スーパークラス)

```
public class Person {  
  
    // -----  
    // フィールド / Fields  
    // -----  
    String name;  
    int birth_year;  
  
    // -----  
    // コンストラクタ / Constructor (初期化 / Initialization)  
    // -----  
    public Person(String n, int y) {  
        name = n;  
        birth_year = y;  
    }  
  
    // -----  
    // メソッド / Method  
    // -----  
    public void talk() {  
        System.out.println("I'm" + name + ".");  
    }  
    public void walk() {  
        /* Walking process */  
    }  
}
```

サブクラス Subclass

- サブクラスは, extendsによって他のクラスを継承したクラスである。
A subclass is a class that inherits another class using extends.



Student.java (Subclass, サブクラス)

```
public class Student extends Person {  
    // フィールド / Fields  
    int id;  
  
    // コンストラクタ / Constructor  
    public Student(String n, int y, int sid) {  
        super(n, y);  
        id = sid;  
    }  
  
    // メソッド / Method  
    @Override  
    public void talk() {  
        super.talk();  
        System.out.println("My student ID is " + id + ".");  
    }  
}
```

Faculty.java (Subclass, サブクラス)

```
public class Faculty extends Person {  
    // フィールド / Fields  
    String course;  
  
    // コンストラクタ / Constructor  
    public Faculty(String n, int y, String c) {  
        super(n, y);  
        course = c;  
    }  
  
    // メソッド / Method  
    @Override  
    public void talk() {  
        super.talk();  
        System.out.println("My teaching class is " + course + ".");  
    }  
}
```

講義内課題 10A In class Assignment 10A

1. “継承の練習”で実行した, **コンパイル**に必要な**コマンド**を回答しなさい.
Answer the **commands** required for **compilation** that were executed in the “Practice Using Inheritance” section.

Commands

```
$ javac SelfIntroduction.java Person.java Student.java Faculty.java
```

2. “継承の練習”で実行した, **実行**に必要な**コマンド**を回答しなさい.
Answer the **commands** required for **execution** that were executed in the “Practice Using Inheritance” section.

Commands

```
$ java SelfIntroduction
```

3. “継承の練習”で得られた, **実行結果**を回答しなさい.
Answer the **execution results** obtained from the “Practice Using Inheritance”.

```
I'm Alice. My student ID is 123456.  
I'm Bob. My student ID is 123457.  
I'm Carol. My teaching class is Java Programming.  
I'm Dave. My teaching class is Electromagnetics.
```

提出方法 Submission method: **Moodle**

提出期限 Submission deadline: **第11回講義の開始まで. By the start of the class 11.**

講義内課題 10B In class Assignment 10B

- “継承の練習”で作成したファイルを修正して、あなたの 氏名 及び 学籍番号 が正しく表示されるように修正しなさい。
Modify the files created in the “Practice Using Inheritance” section so that your name and student ID are displayed correctly.
- 修正したファイルのみを、ファイル名を変更せずにそのままアップロードして下さい。
Upload **ONLY** the modified files **without changing** their file names.

Student.java (Subclass, サブクラス)

```
public class Student extends Person {  
    // フィールド / Fields  
    int id;  
  
    // コンストラクタ / Constructor  
    public Student(String n, int y, int sid) {  
        super(n, y);  
        id = sid;  
    }  
  
    // メソッド / Method  
    @Override  
    public void talk() {  
        super.talk();  
        System.out.println("My student ID is " + id + ".");  
    }  
}
```



Student.java (Subclass, サブクラス)

```
public class Student extends Person {  
    // フィールド / Fields  
    String id;  
  
    // コンストラクタ / Constructor  
    public Student(String n, int y, String sid) {  
        super(n, y);  
        id = sid;  
    }  
  
    // メソッド / Method  
    @Override  
    public void talk() {  
        super.talk();  
        System.out.println("My student ID is " + id + ".");  
    }  
}
```

SelfIntroduction.java

```
public class SelfIntroduction {  
    public static void main(String[] args) {  
  
        Student st1 = new Student("Alice", 2005, 123456);  
        Student st2 = new Student("Bob", 2006, 123457);  
  
        Faculty fc1 = new Faculty("Carol", 1980, "Java Programming");  
        Faculty fc2 = new Faculty("Dave", 1980, " Electromagnetics");  
  
        st1.talk();  
        st2.talk();  
  
        fc1.talk();  
        fc2.talk();  
    }  
}
```



SelfIntroduction.java

```
public class SelfIntroduction {  
    public static void main(String[] args) {  
  
        Student st1 = new Student("Your Name", 2005, "1w123456");  
        Student st2 = new Student("Alice", 2006, "1w123457");  
  
        Faculty fc1 = new Faculty("Carol", 1980, "Java Programming");  
        Faculty fc2 = new Faculty("Dave", 1980, " Electromagnetics");  
  
        st1.talk();  
        st2.talk();  
  
        fc1.talk();  
        fc2.talk();  
    }  
}
```

提出方法 Submission method: **Moodle**

提出期限 Submission deadline: **第11回講義の開始まで. By the start of the class 11.**

講義内課題 10C In class Assignment 10C

1. 講義内課題 9B の回答例として示した下記3ファイルを作成しなさい。作成するプログラムは、回答例と同じ内容・動作になるようにして下さい。日本語/英語の両方で重複して記載されている コメント/print()/println() は、どちらか一方の言語のみで作成しても問題ありません。

Create the following three files shown as the sample answer for In-class Assignment 9B. The created program must have the same content and behavior as the sample answer. For comments/print()/println() written redundantly in both Japanese and English, you may write only one of the two languages.

- 1. MonsterBattleGame_WithClass_2.java
- 2. GameCharacter.java
- 3. BattleProcess.java

2. GameCharacterクラスを継承するサブクラスとしてPlayerクラスとMonsterクラスを作成しなさい。作成したサブクラスを用いて、MonsterBattleGame_WithClass_2.javaが正しく動作するようにプログラムを修正しなさい。作成・修正した最終版のファイルをZipファイルに圧縮したうえでMoodleに提出して下さい。また修正した箇所と内容を説明しなさい。

Create the Player class and the Monster class as subclasses that extend the GameCharacter class. Modify MonsterBattleGame_WithClass_2.java so that the program works correctly using the created subclasses.

Compress the final versions of the created and modified files into a Zip file, and submit the Zip file on Moodle. Also explain which parts you modified and how you modified them.

提出ファイル名 Submission file name : Assignment10C_[Student ID].zip

提出方法 Submission method : Moodle

提出期限 Submission deadline : 第11回講義の開始まで。 By the start of the class 11.

講義内課題 10C In class Assignment 10C

MonsterBattleGame_WithClass_2.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class MonsterBattleGame_WithClass_2 {

    public static void main(String args[]){ // プログラムの実行開始地点 / The starting point of program execution

        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // Enter入力を受け取るための準備 / Prepare for reading Enter input
        Random rand = new Random(999); // 乱数生成処理の初期化 / Initialize random number generation

        // プレイヤー配列の生成 / Create the player array
        Player[] players = new Player[1];

        // モンスター配列の生成 / Create the monster array
        Monster[] monsters = new Monster[3];

        // インスタンスの生成と参照 / Creating and referring to instances
        players[0] = new Player("Player0", 300, 60, 20); // プレイヤー0を生成 / Create the player0
        monsters[0] = new Monster("Monster0", 100, 20, 10); // モンスター0を生成 / Create Monster0
        monsters[1] = new Monster("Monster1", 100, 20, 10); // モンスター1を生成 / Create Monster1
        monsters[2] = new Monster("Monster2", 100, 20, 10); // モンスター2を生成 / Create Monster2

        // ゲーム情報の表示 / Display the game information
        System.out.println("~~~~~");
        System.out.println("        MonsterBattleGame_WithClass_2        ");
        System.out.println("~~~~~");
        for (int i = 0; i < players.length; i++) { // 全プレイヤーについて繰り返す / Repeat for all players
            players[i].print(); // プレイヤー情報の表示 / Display player information
        }
        for (int i = 0; i < monsters.length; i++) { // 全モンスターについて繰り返す / Repeat for all monsters
            monsters[i].print(); // モンスター情報の表示 / Display monster information
        }

        // バトル進行 / Battle control
        System.out.println("# バトル開始 / Battle start");
        while (true) {

            System.out.println("# Enterを入力して攻撃 / Press Enter to attack");
            scan.nextLine(); // プレイヤーの入力待ち / Wait for player input

            // 各プレイヤーが各モンスターを攻撃 / Each player attacks each monster
            BattleProcess.performAttacks(players, monsters, rand);

            // モンスターの戦闘不能判定 / Check whether monsters are defeated
            if (BattleProcess.allDefeated(monsters)) { // 全モンスターが戦闘不能の場合 / If all monsters are defeated
                BattleProcess.printWinner(players[0]); // プレイヤーの勝利 / Players win
                break; // ループを抜ける / Exit the loop
            }

            // 各モンスターが各プレイヤーを攻撃 / Each monster attacks each player
            BattleProcess.performAttacks(monsters, players, rand);

            // プレイヤーの戦闘不能判定 / Check whether players are defeated
            if (BattleProcess.allDefeated(players)) { // 全プレイヤーが戦闘不能の場合 / If all players are defeated
                BattleProcess.printWinner(monsters[0]); // モンスターの勝利 / Monsters win
                break; // ループを抜ける / Exit the loop
            }
        }

        // 終了処理 / Exit handling
        scan.close(); // Scannerを閉じる / Close the Scanner
    }
}
```

Original program

```
// プレイヤー配列の生成 / Create the player array
GameCharacter[] players = new GameCharacter[1];

// モンスター配列の生成 / Create the monster array
GameCharacter[] monsters = new GameCharacter[3];

// インスタンスの生成と参照 / Creating and referring to instances
players[0] = new GameCharacter("Player0", 300, 60, 20); // プレイヤー0を生成 / Create the player0
monsters[0] = new GameCharacter("Monster0", 100, 20, 10); // モンスター0を生成 / Create Monster0
monsters[1] = new GameCharacter("Monster1", 100, 20, 10); // モンスター1を生成 / Create Monster1
monsters[2] = new GameCharacter("Monster2", 100, 20, 10); // モンスター2を生成 / Create Monster2
```



講義内課題 10C In class Assignment 10C

Monster.java

今回の課題の範囲では、ただGameCharacterクラスを継承しただけで、各クラスにおける独自実装は含めていない。

```
// =====  
// Monsterクラス / Monster class  
// =====  
public class Monster extends GameCharacter {  
  
    // コンストラクタ / Constructor  
    public Monster(String character_name, int character_hp, int character_attack_power, int character_attack_fails_rate) {  
        super(character_name, character_hp, character_attack_power, character_attack_fails_rate);  
    }  
  
}
```

Player.java

In the scope of this assignment, the classes only inherit from GameCharacter class and do not include any class-specific implementations.

```
// =====  
// Playerクラス / Player class  
// =====  
public class Player extends GameCharacter {  
  
    // コンストラクタ / Constructor  
    public Player(String character_name, int character_hp, int character_attack_power, int character_attack_fails_rate) {  
        super(character_name, character_hp, character_attack_power, character_attack_fails_rate);  
    }  
  
}
```

GameCharacter.java と BattleProcess.javaは修正不要

GameCharacter.java and BattleProcess.java do not need to be modified.



例外処理・ファイル操作

Exception Handling, File I/O

例外処理 Exception Handling

例外とは、プログラム実行中に発生するエラーのこと。

try-catch文を入れることで、例外(エラー)の発生箇所が特定しやすくなる。

An exception is an error that occurs during program execution. By using a try-catch statement, it becomes easier to identify where an exception (error) occurs.

ExceptionHandling.java

```
public class ExceptionHandling {  
    public static void main(String args[]){  
        MethodsIncludingError.causeArrayExceptions(5);  
    }  
}
```

1. 左のプログラムを作成して実行しなさい。

Write and execute the program shown on the left.

MethodsIncludingError.java

```
public class MethodsIncludingError {  
    public static void causeArrayExceptions(int array_max) {  
        try {  
            int[] array = new int[array_max];  
            for(int i=0; i <= array_max; i++) {  
                array[i] = i;  
                System.out.println(array[i]);  
            }  
        } catch (Exception e) {  
            e.printStackTrace(); // ArrayIndexOutOfBoundsException  
            System.exit(1);  
        }  
    }  
}
```

2. どのようなエラーが発生したか読み取りなさい。

Read what kind of error occurred.

Commands

```
$ javac ExceptionHandling.java MethodsIncludingError.java  
$ java ExceptionHandling
```

例外処理 Exception Handling

try-catch文は必要最小限の範囲に入れることが原則

→ 例外の発生箇所が特定しやすくなる

→ 例外発生時の処理を追加できる

As a general rule, a try-catch statement should be placed around the minimum necessary range of code.

→ This makes it easier to identify where an exception occurs.

→ This also makes it possible to add specific processing when an exception occurs.

ExceptionHandling.java

```
public class ExceptionHandling {
    public static void main(String args[]){
        if (args.length < 2) {
            System.out.println("Please enter two values as command line args.");
            return;
        }
        int response = MethodsIncludingError.causeArithmeticException(args[0], args[1]);
        System.out.println("causeArithmeticException response: " + response);
    }
}
```

MethodsIncludingError.java

```
public class MethodsIncludingError {
    public static int causeArithmeticException(String input1, String input2) {
        int num1;
        int num2;

        // 引数を整数に変換 / Convert the arguments to an integer
        try {
            num1 = Integer.parseInt(input1);
            num2 = Integer.parseInt(input2);
        } catch (NumberFormatException e) {
            e.printStackTrace(); // NumberFormatException
            return 1;
        }
        // 割り算を実行 / Perform division
        try {
            int result = num1 / num2;
            System.out.println(num1 + "/" + num2 + " = " + result);
        } catch (ArithmeticException e) {
            e.printStackTrace(); // ArithmeticException
            return 2;
        }
        return 0;
    }
}
```

Commands

```
$ javac ExceptionHandling.java MethodsIncludingError.java
```

```
$ java ExceptionHandling 10 3
10/3 = 3
causeArithmeticException response: 0
```

```
$ java ExceptionHandling 10 0
java.lang.ArithmeticException: / by zero
    at MethodsIncludingError.causeArithmeticException(MethodsIncludingError.java:29)
    at ExceptionHandling.main(ExceptionHandling.java:10)
causeArithmeticException response: 2
```

```
$ java ExceptionHandling 10 aaa
java.lang.NumberFormatException: For input string: "aaa"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:662)
    at java.base/java.lang.Integer.parseInt(Integer.java:778)
    at MethodsIncludingError.causeArithmeticException(MethodsIncludingError.java:22)
    at ExceptionHandling.main(ExceptionHandling.java:10)
causeArithmeticException response: 1
```

エラー箇所の特定が容易

Easier identification of the error location

ファイル入出力 File-IO

ファイル操作を行う場合は、特に例外が発生しやすく、try-catch文が役立つ。

When working with files, exceptions are particularly likely to occur, making try-catch statements useful.

FileCopy.java

```
import java.io.*;
import java.util.Scanner;

public class FileCopy {
    public static void main(String[] args) {

        // 入力ファイルを出力ファイルへコピーする / Copy the input file to the output file
        try {
            File file = new File(args[0]); // 入力ファイルを指定する / Specify the input file
            Scanner sc = new Scanner(file); // 入力ファイルを読み込む / Read the input file

            FileWriter fw = new FileWriter(args[1]); // 出力ファイルを指定する / Specify the output file
            BufferedWriter bw = new BufferedWriter(fw); // 出力を準備する / Prepare output
            PrintWriter pw = new PrintWriter(bw); // ファイルへ書き込む / Write to the file

            while (sc.hasNextLine()) {
                String line = sc.nextLine(); // 1行読み込む / Read one line
                System.out.println(line); // 画面に表示する / Display the line
                pw.println(line); // ファイルに書き込む / Write the line to the file
            }

            sc.close(); // 入力ファイルを閉じる / Close the input file
            pw.close(); // 出力ファイルを閉じる / Close the output file
        } catch (Exception e) {
            e.printStackTrace(); // 例外の内容を表示する / Display the exception
            System.exit(1); // プログラムを終了する / Exit the program
        }
    }
}
```

Commands

```
$ javac FileCopy.java
$ java FileCopy FileCopy.java test.out

public class FileCopy {
    public static void main(String[] args) {

        // 入力ファイルを出力ファイルへコピーする / Copy the input file to the output file
        try {
            File file = new File(args[0]); // 入力ファイルを指定する / Specify the input file
            Scanner sc = new Scanner(file); // 入力ファイルを読み込む / Read the input file

            FileWriter fw = new FileWriter(args[1]); // 出力ファイルを指定する / Specify the output file
            BufferedWriter bw = new BufferedWriter(fw); // 出力を準備する / Prepare output
            PrintWriter pw = new PrintWriter(bw); // ファイルへ書き込む / Write to the file

            while (sc.hasNextLine()) {
                String line = sc.nextLine(); // 1行読み込む / Read one line
                System.out.println(line); // 画面に表示する / Display the line
                pw.println(line); // ファイルに書き込む / Write the line to the file
            }

            sc.close(); // 入力ファイルを閉じる / Close the input file
            pw.close(); // 出力ファイルを閉じる / Close the output file
        } catch (Exception e) {
            e.printStackTrace(); // 例外の内容を表示する / Display the exception
            System.exit(1); // プログラムを終了する / Exit the program
        }
    }
}

$ ls
FileCopy.class FileCopy.java test.out
$ cat test.out

public class FileCopy {
    public static void main(String[] args) {

        // 入力ファイルを出力ファイルへコピーする / Copy the input file to the output file
        try {
            File file = new File(args[0]); // 入力ファイルを指定する / Specify the input file
            Scanner sc = new Scanner(file); // 入力ファイルを読み込む / Read the input file

            FileWriter fw = new FileWriter(args[1]); // 出力ファイルを指定する / Specify the output file
            BufferedWriter bw = new BufferedWriter(fw); // 出力を準備する / Prepare output
            PrintWriter pw = new PrintWriter(bw); // ファイルへ書き込む / Write to the file

            while (sc.hasNextLine()) {
                String line = sc.nextLine(); // 1行読み込む / Read one line
                System.out.println(line); // 画面に表示する / Display the line
                pw.println(line); // ファイルに書き込む / Write the line to the file
            }

            sc.close(); // 入力ファイルを閉じる / Close the input file
            pw.close(); // 出力ファイルを閉じる / Close the output file
        } catch (Exception e) {
            e.printStackTrace(); // 例外の内容を表示する / Display the exception
            System.exit(1); // プログラムを終了する / Exit the program
        }
    }
}
```

課題が終われば退出可能です。
TAさんに自身の座席番号を伝えてからご退出下さい。

You can leave once you have completed the assignment. Please inform
a teaching assistant (TA) of your seat number before leaving.

講義内課題

In class Assignment

Please do not include your CD (check digit)
in the student ID part of the filename.

Example:

Filename Format: Assignment9A_[Student ID].java

Student ID: 1w123456-7

Filename: Assignment9A_1w123456.java

講義内課題 11A In class Assignment 11A

下に示した ExceptionHandling.java, MethodsIncludingError.java (現在は例外発生なし) を変更し, 講義で扱った例とは別の箇所で, 配列に関する例外が発生するようにしなさい. ただし, 変更文字数が少ないほど高得点とする. 作成・修正した最終版のファイルを[Zipファイルに圧縮](#)したうえでMoodleに提出して下さい. また, [Moodleの自由記述欄](#)に, 変更前後の差分と, 例外の内容を明記しなさい.

Modify the following ExceptionHandling.java and MethodsIncludingError.java files to produce an array-related exception at a location different from the lecture example. The files currently do not produce any exceptions. Higher scores will be given for fewer character changes.

Compress the final versions of the files you created or modified into a [ZIP file](#) and submit it to Moodle. Also, in the [text field in Moodle](#), clearly state the differences before and after your modification and describe the exception that occurred.

ExceptionHandling.java

```
public class ExceptionHandling {
    public static void main(String args[]){
        MethodsIncludingError.causeArrayExceptions(5);
    }
}
```

Commands

```
$ javac ExceptionHandling.java MethodsIncludingError.java
$ java ExceptionHandling
```

MethodsIncludingError.java

```
public class MethodsIncludingError {
    public static void causeArrayExceptions(int array_max) {
        try {
            int[] array = new int[array_max];
            for(int i=0; i < array_max; i++) {
                array[i] = i;
                System.out.println(array[i]);
            }
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

提出ファイル名 **Submission file name** : Assignment11A_[Student ID].zip

提出方法 **Submission method** : Moodle (ファイルアップロード 及び テキスト入力 File Upload and Text Input)

提出期限 **Submission deadline** : 第12回講義の開始まで. By the start of the class 12.

講義内課題 11B In class Assignment 11B

FileCopy.java を変更せずに、例外を発生させなさい。

実行したコマンド及び発生した例外、その例外がなぜ発生したか説明しなさい。

Without modifying FileCopy.java, cause an exception.

Explain the command you executed, the exception that occurred, and why the exception occurred.

FileCopy.java

```
import java.io.*;
import java.util.Scanner;

public class FileCopy {
    public static void main(String[] args) {

        // 入力ファイルを出力ファイルへコピーする / Copy the input file to the output file
        try {
            File file = new File(args[0]); // 入力ファイルを指定する / Specify the input file
            Scanner sc = new Scanner(file); // 入力ファイルを読み込む / Read the input file

            FileWriter fw = new FileWriter(args[1]); // 出力ファイルを指定する / Specify the output file
            BufferedWriter bw = new BufferedWriter(fw); // 出力を準備する / Prepare output
            PrintWriter pw = new PrintWriter(bw); // ファイルへ書き込む / Write to the file

            while (sc.hasNextLine()) {
                String line = sc.nextLine(); // 1行読み込む / Read one line
                System.out.println(line); // 画面に表示する / Display the line
                pw.println(line); // ファイルに書き込む / Write the line to the file
            }

            sc.close(); // 入力ファイルを閉じる / Close the input file
            pw.close(); // 出力ファイルを閉じる / Close the output file
        }
        catch (Exception e) {
            e.printStackTrace(); // 例外の内容を表示する / Display the exception
            System.exit(1); // プログラムを終了する / Exit the program
        }
    }
}
```

提出方法 Submission method : Moodle (テキスト入力 Text Input)

提出期限 Submission deadline : 第12回講義の開始まで。 By the start of the class 12.

講義内課題 11C In class Assignment 11C

1. 次の資料を参考に、第1回講義で確認したGUI機能(xeyes)が利用できることを再確認して下さい。また、xeyesが動作していることを示すスクリーンショットを提出しなさい。

Using the following document as a reference, please reconfirm that you can use the GUI functions (xeyes) covered in the class-1. Also, submit a screenshot showing that xeyes is running.

<https://waseda.box.com/v/virtualmachine>

2. 右に示すHina01.javaを作成・実行して、GUI出力のスクリーンショットを提出しなさい。理工Linuxのターミナル上の学籍番号部分を含めること。

Create and run Hina01.java shown on the right, and submit a screenshot of the GUI output. The screenshot must include the student ID area displayed in the Riko-Linux terminal.

```
FileWriter fw = new FileWriter(args[1]); // 出力ファイルを指定する / Specify the output file
BufferedWriter bw = new BufferedWriter(fw); // 出力を準備する / Prepare output
PrintWriter pw = new PrintWriter(bw); // ファイルへ書き込む / Write to the file

while (sc.hasNextLine()) {
    String line = sc.nextLine(); // 一行を読み取る / Read one line
    System.out.println(line); // 画面に出力する / Display the line
    pw.println(line); // ファイルへ書き込む / Write the line to the file
}

sc.close(); // 入力ファイルを閉じる / Close the input file
pw.close(); // 出力ファイルを閉じる / Close the output file

catch (Exception e) {
    e.printStackTrace(); // 例外の内容を表示する / Display the exception
    System.exit(1); // プログラムを終了する / Exit the program
}
```

Hina01.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Hina01 extends JFrame{

    public Hina01(){
        setSize(800,500);
        setTitle("Java Programming");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        MyJPanel myJPanel= new MyJPanel();
        Container c = getContentPane();
        c.add(myJPanel);
        setVisible(true);
    }

    public static void main(String[] args){
        new Hina01();
    }

    public class MyJPanel extends JPanel{
        public MyJPanel(){
        }

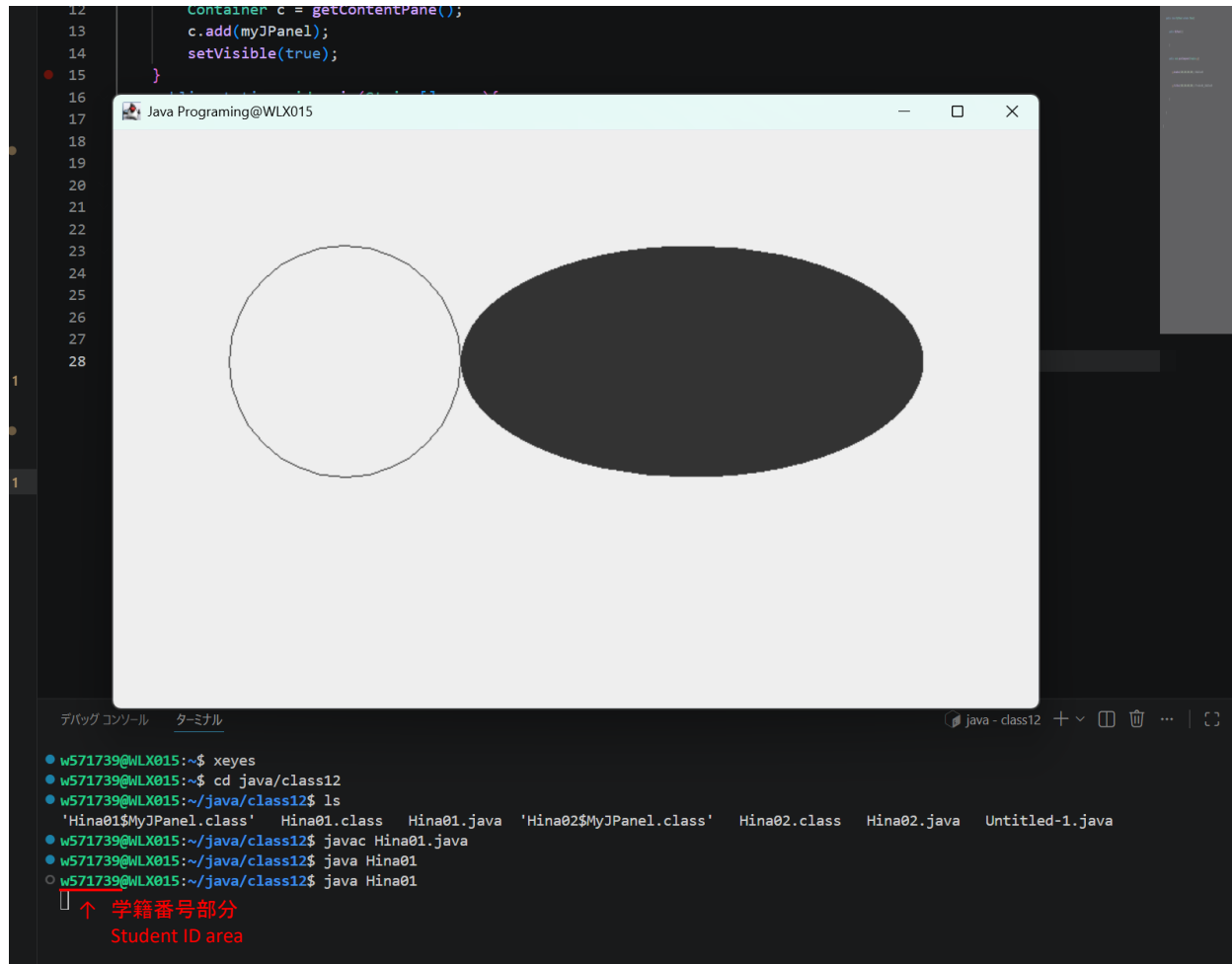
        public void paintComponent(Graphics g){
            g.drawOval(100,100,200,200); // 楕円を描く
            g.fillOval(300,100,400,200); // 塗りつぶした楕円を描く
        }
    }
}
```

提出ファイル名 **Submission file name** : Assignment11C-1_[Student ID].png or .jpg and Assignment11C-2_[Student ID].png or .jpg

提出方法 **Submission method** : Moodle (ファイルアップロード File Upload)

提出期限 **Submission deadline** : 第12回講義の開始まで。 By the start of the class 12.

講義内課題 11C In class Assignment 11C



提出ファイル名 **Submission file name** : Assignment11C-1_[Student ID].png or .jpg and Assignment11C-2_[Student ID].png or .jpg

提出方法 **Submission method** : Moodle (ファイルアップロード File Upload)

提出期限 **Submission deadline** : 第12回講義の開始まで. By the start of the class 12.